Lab 1 – Språkteknologi (5DV218)

Name: Michail Pettas

Everything below was run with spaCy 3.8. I used en_core_web_sm for English (and en_core_web_md in Task 4 to compare) and sv_core_news_sm for the Swedish part. All numbers and tags come from my script lab1.py, which is in the separate code PDF, and the full output is pasted at the end.

1. Tokenization with spaCy

spaCy splits the text into **46 tokens**:

```
On | Jan. | 15 | , | 2024 | , | Dr. | Smith | 's | research | on | AI | - | generated | art | sparked | debates | on | ' | true | ' | creativity | . | While | some | critics | call | it | mere | imitation | , | others | see | it | as | a | $ | 10 | million | revolution | in | human | – | AI | collaboration | !
```

As far as I understand it, the tokenizer first splits on whitespace and then goes through each piece, cutting off prefixes and suffixes and splitting on infixes, while checking a list of special cases (exceptions). All of these steps can be seen in the output:

- **Abbreviations:** Jan. and Dr. keep their period because they are in the exception list, so the period isn't treated as the end of a sentence.
- **Punctuation:** the commas after 15 and 2024, the final . and ! and the quotes around 'true' all end up as separate tokens.
- **Clitics:** Smith's is split into Smith and 's.
- **Infixes:** AI-generated and human–AI both become three tokens, with the hyphen or en dash in the middle as a token of its own.
- **Currency:** $10 is split into $ and 10, and $ gets is_currency=True.
- Nothing is lost in the split, since every token stores whether it had a space after it (token.whitespace_). So the original string can be put back together exactly.

2. Filtering tokens (no numbers, number words or stop words)

I kept a token if not token.like_num and not token.is_stop, which leaves **28 of 46 tokens**:

```
Jan. , , Dr. Smith research AI - generated art sparked debates ' true ' creativity . critics mere imitation , $ revolution human – AI collaboration !
```

Removed because	Tokens
like_num (digits)	15, 2024, 10
like_num (number word)	million
is_stop	On, 's, on, on, While, some, call, it, others, see, it, as, a, in

like_num works for number words as well as digits. I tested it on "one two first third ten hundred million" and got True for all of them, ordinals included, so this one attribute covers both number conditions in the task. Something I didn't expect was that call and see are on spaCy's stop word list.

The filter as the task describes it lets punctuation and $ through, since they are neither numbers nor stop words. If is_punct and is_currency are removed as well, only the **18 content tokens** are left:

```
Jan. Dr. Smith research AI generated art sparked debates true creativity critics mere imitation revolution human AI collaboration
```

3. spaCy tokenizer vs. Python's split()

I used this sentence: "Hello, world! Isn't this state-of-the-art? I'm sure it is. Don't you think so, Dr. Brown? We'll see: prices rose 3.5% in the U.S." split() returns 23 tokens and spaCy 41.

Phenomenon	Input	split()	spaCy
Punctuation	world! / so, / is.	world! / so, / is.	world ! / so , / is .
	Brown?	Brown?	Brown ?
	Dr. / U.S. / 3.5%	Dr. / U.S. / 3.5%	Dr. / U.S. / 3.5 %
Contractions	Isn't / Don't	Isn't / Don't	Is n't / Do n't (norm of n't = not)
	I'm / We'll	I'm / We'll	I 'm / We 'll ('m: lemma be, AUX; 'll: will, MD)
Hyphenated words	state-of-the-art?	state-of-the-art? (one token, including the ?)	state - of - the - art ?

Punctuation. split() only splits on whitespace, so punctuation stays stuck to the word before it. is. and is become two different words, and Brown? is a word of its own too. spaCy separates commas, periods, question marks and colons, but leaves the period alone when it is part of a word (Dr., U.S.) or a decimal number (3.5).

Contractions. With split(), I'm and don't stay as one unit each. spaCy splits them into the two words they stand for, so each part gets its own lemma and tag. That way n't can be recognised as a negation (not) and 'm as a form of be.

Hyphenated words. split() keeps the whole compound as one token. spaCy splits at every hyphen between letters and tags the hyphens HYPH. There is no whitespace after the hyphens, so the compound can be merged back into one token with the retokenizer if that's needed.

Which is more suitable for POS tagging and parsing? spaCy's tokenizer. The tagger and parser give one tag and one head to each syntactic word, and the models were trained on data (OntoNotes, Penn Treebank conventions) that is tokenized the same way spaCy does it. To see what happens otherwise, I gave the tagger the split() tokens as a pre-tokenized Doc:

Tokens from	Tags for "I'm sure it is. Don't you think so, Dr. Brown?"
spaCy	I/PRP 'm/VBP sure/JJ it/PRP is/VBZ ./. Do/VBP n't/RB you/PRP think/VB so/RB ,/, Dr./NNP Brown/NNP ?/.
split()	I'm/PRP sure/JJ it/PRP is./UH Don't/RB you/PRP think/VBP so,/NN Dr./NNP Brown?/NN

With split() tokens, I'm can only get one tag, so the verb 'm disappears, and the same happens to the auxiliary in Don't. is. gets tagged as an interjection, and so, and Brown? as common nouns, since the model has never seen those strings. The sentence-final punctuation also stops being a separate token, and that is what marks where a sentence ends. I'd say split() is only OK for rough word counts on clean text.

4. UPOS vs. XPOS

UPOS (token.pos_) is the Universal Dependencies tagset. It has 17 tags and they are the same for all languages. XPOS (token.tag_) is specific to the language, which for English means the Penn Treebank tagset with around 50 tags, and it also encodes some morphology (tense, person, number, wh-words). Several XPOS tags map to the same UPOS tag, but as the table shows it isn't always a straight lookup:

Token(s)	UPOS	XPOS	Comment
jumps / saw / pick / flies	VERB	VBZ / VBD / VB / VBZ	XPOS keeps tense and person; UPOS gives only "verb".

does	AUX	VBZ	Same XPOS as jumps, but UPOS marks its function as an auxiliary.
fox / dog / binoculars	NOUN	NN / NN / NNS	Number exists only in XPOS.
over / on / with / like	ADP	IN	PTB IN covers prepositions and subordinating conjunctions.
why	SCONJ	WRB	The mapping is context-dependent. WRB becomes ADV or SCONJ, and SCONJ is questionable here.
it / She	PRON	PRP	
that (that poor dog)	DET	DT	Demonstrative here. In other contexts that is SCONJ/IN or PRON/WDT.

How context affects the tags

Sentence 1: The quick brown fox jumps over the lazy dog, but why does it always pick on that poor dog? On its own, jumps could be a plural noun (the jumps), but here it comes right after a full subject NP, so it's tagged VBZ. does is AUX and not VERB because a main verb (pick) comes after it, and that is a determiner because it's followed by an adjective and a noun. In pick on, the tagger took on as a preposition (ADP/IN) instead of a particle (RP). For this phrasal verb I think both readings can be defended.

Sentence 2: She saw the man with binoculars. saw could also be a noun (the tool), but after a pronoun subject it gets VBD. The ambiguity in this sentence is structural rather than lexical. The POS tags are identical for both readings, and the question is whether with binoculars attaches to the verb (she used binoculars) or to the noun (the man had them). The two models actually disagree here: en_core_web_sm attaches with to man, while en_core_web_md attaches it to saw. POS tags alone can't settle this, you need the parse and some world knowledge.

Sentence 3: Time flies like an arrow; fruit flies like a banana. Both clauses look the same on the surface (N N-s like DT N), but they have different structures:

	Time	**flies**	**like**	**fruit**	**flies**	**like**
Intended	NN	VBZ	IN	NN	NNS (insects)	VBP (verb "enjoy")
en_core_web_sm	NN	VBZ	IN	NN	NNS	IN
en_core_web_md	NN	VBZ	IN	NN	VBZ	IN

Both models get the first clause right and the second one wrong. The small model does read fruit flies as a noun phrase, but then it tags like as a preposition, so the clause ends up without a verb and the parser just hangs flies on the first flies as a conj (Figure 1). The medium model copies the structure of the first clause instead, so fruit "flies" the way a banana does. To get the intended reading you have to know that fruit flies are insects and that bananas are food, and the tagger has no access to that kind of knowledge.

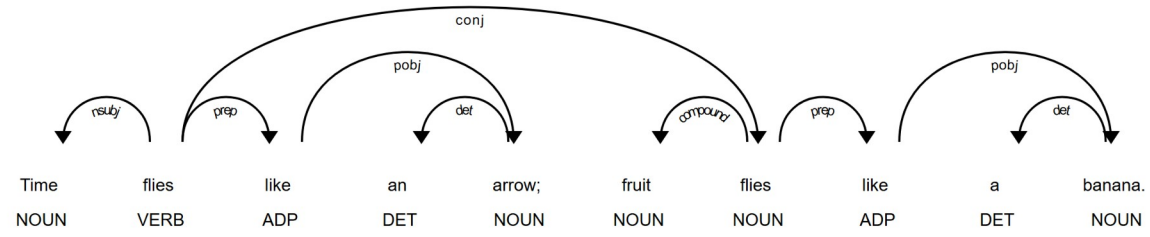


Figure 1. en_core_web_sm parse of "Time flies like an arrow; fruit flies like a banana." The second clause has no verb.

Why rule-based approaches fail. A rule-based tagger has a lexicon and some local rules, e.g. "the word after a determiner is a noun", "a word ending in -s after a noun is a verb", or simply "pick the most frequent tag". The sentences above break all of these. A rule that gets time flies right (flies as a verb) gets fruit flies wrong, and a rule that treats N+N as a compound has the opposite problem. The most frequent tag for like (IN) is wrong in the second clause, saw and jumps can both be nouns or verbs, and the attachment of with binoculars can't be decided from tags at all. Getting these right depends on context further away than a fixed window and on what the sentence means, and it's not possible to write rules for every case without them starting to contradict each other. Statistical taggers generalise better, but the results above show that they fail too when the ambiguity can only be resolved with world knowledge.

5. POS tagging for travel queries

For each query the program finds the verb with lemma fly and decides its time reference from the XPOS tag (tag_), the morphology (morph), the auxiliaries (dep_ == "aux") and, for an xcomp, its head:

#	Verb group	Tags and morphology	Time reference	Booking?
1	flew (also haven't received)	flew VERB/VBD, Tense=Past; 3 days ago (advmod)	Past	No
2	have flown (also forgot)	have AUX/VBP (aux) + flown VBN, Aspect=Perf, Tense=Past; forgot VBD	Past (present perfect, completed)	No
3	'm flying	'm AUX/VBP (aux) + flying VBG, Aspect=Prog, Tense=Pres; next/JJ → week/NN npadvmod of flying	Future (planned, present progressive with a future time adverbial)	Yes
4	need to fly	need VERB/VBP ROOT → fly VB (VerbForm=Inf) xcomp, to PART/TO aux; next Tuesday in the second sentence	Future (intent)	Yes
5	will fly	will AUX/MD (aux) + fly VB, VerbForm=Inf; next week	Future	Yes
6	will fly	will AUX/MD (aux) + fly VB, VerbForm=Inf; next Monday (PROPN, npadvmod)	Future	Yes

Selected queries: 3, 4, 5 and 6.

- Past: VBD, or have + VBN (Tense=Past, Aspect=Perf). That covers queries 1 and 2, which are about flights that have already happened (a missing bill and a laptop left on the plane).
- Future: the modal will/MD + VB (queries 5 and 6).
- Intent: to/TO + VB as an xcomp of need (query 4).
- Query 3 was the hardest one. Formally it's present tense (VBP + VBG, Tense=Pres), so going only by tags and morphology it would count as present. The future meaning comes from next week, which is attached to the verb as npadvmod, so the program has to look at the dependency relations and not just the tags.

Query 6 is about Vienna and not Rome, but it is still a planned future flight, so I counted it as a booking request too.

6. Dependency tree

Sentence: *"Although she was tired, she finished her work before midnight."*

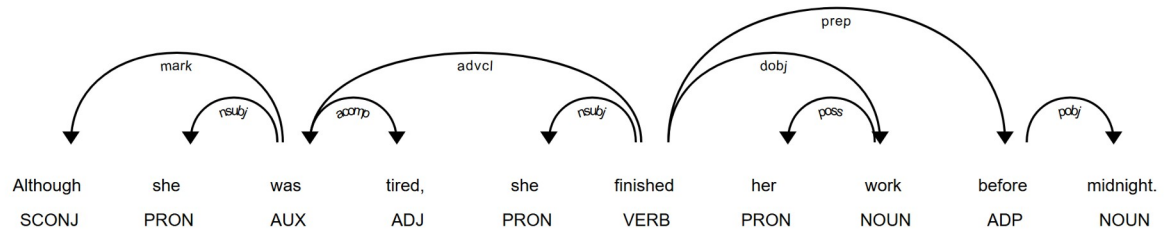


Figure 2. spaCy dependency tree (displacy, en_core_web_sm).

Token	**POS**	**dep**	**head**
Although	SCONJ	mark	was
she	PRON	nsubj	was
was	AUX	advcl	finished
tired	ADJ	acomp	was
she	PRON	nsubj	finished
finished	VERB	ROOT	–
her	PRON	poss	work
work	NOUN	dobj	finished
before	ADP	prep	finished
midnight	NOUN	pobj	before

This is the tree as my program prints it (punctuation included):

```
finished [ROOT, VERB]
    was [advcl, AUX]
        Although [mark, SCONJ]
        she [nsubj, PRON]
        tired [acomp, ADJ]
    , [punct, PUNCT]
    she [nsubj, PRON]
    work [dobj, NOUN]
        her [poss, PRON]
    before [prep, ADP]
        midnight [pobj, NOUN]
    . [punct, PUNCT]
```

Subject and object of the main clause. The main verb (ROOT) is *finished*. Its subject is the second *she* (nsubj) and its object is *her work* (dobj on *work*, with *her* as poss). The first *she* is only the subject of the subordinate clause.

How "although" connects the clauses. *Although* doesn't attach to the main verb directly. It is a mark (subordinating conjunction) on *was*, the head of the subordinate clause *Although she was tired*, and that clause is attached to *finished* with the relation advcl (adverbial clause modifier). So the path goes *Although* → mark → *was* → advcl → *finished*. The mark is what makes the clause concessive, and advcl makes the whole clause a modifier of the main clause.

The English model uses ClearNLP-style labels, so the copula *was* is the head of *she was tired* and *tired* is acomp. In Universal Dependencies it would be the other way around, with *tired* as the head and *was* labelled cop. The Swedish model in Task 9 does it the UD way.

7. Syntax and passive voice

I count a sentence as passive if the main verb (a past participle, VBN) has an nsubjpass (the passive subject, i.e. the patient) and an auxpass (a form of be). The agent is the pobj of the preposition labelled agent (by), and the tense comes from the tag and morphology of the passive auxiliary.

Sentence	Voice	Patient (nsubjpass)	Agent (agent → pobj)	Tense
John bought a ticket for the concert yesterday.	Active (nsubj + VBD + dobj)			
Alice helped her friend with the homework.	Active			
The team completed the project on time.	Active			
The problem was solved quickly by Mark.	Passive	The problem	Mark	Past (was VBD + solved VBN)
A new product was launched by the company last week.	Passive	A new product	the company	Past (was VBD + launched VBN)
Flowers were planted in the garden by Sarah.	Passive	Flowers	Sarah	Past (were VBD + planted VBN)
The decision was made yesterday.	Passive	The decision	not expressed (agentless passive)	Past (was VBD + made VBN)

All four passive sentences have the auxiliary tagged VBD with Tense=Past, so they are all simple past. Present would have been is/are (VBZ/VBP), and future will be + VBN (with MD as aux). The last sentence has no by-phrase and so no token labelled agent, but the parser still recognises it as passive and the agent is just left implicit. *In the garden* in the sixth sentence is an ordinary prep, so the program doesn't mistake it for the agent.

8. DP and POS tagging: Alisa flying from Rome

For every verb with lemma fly (UPOS VERB) the program checks two things. Alisa has to be its nsubj, or a conj of the nsubj so that coordinated subjects like Alisa and Luk are handled, and the verb needs a prep child from whose pobj is Rome.

#	Verb (tag, morph)	Subject(s) of fly	from → pobj	Role of Alisa	Match
1	flew VBD, Tense=Past	Alisa (nsubj, PROPN/NNP)	Rome	nsubj → flew	Yes
2	have flown VBN, Aspect=Perf	parents (nsubj)	Athens	dobj → visit	No
3	will fly MD + VB, VerbForm=Inf	Alisa (nsubj) + Luk (conj)	Rome	nsubj → fly	Yes
4	are flying VBP + VBG, Aspect=Prog	They (nsubj, PRON)	New York	not present	No
5	had flown VBD + VBN, Aspect=Perf	John (nsubj)	Rome	nsubj → received	No

Matching sentences: 1 and 3.

- **Sentence 1.** Alisa (PROPN/NNP) is the nsubj of flew (VERB/VBD, past), and from (prep) → Rome (pobj) hangs off flew.
- **Sentence 3.** Alisa is the nsubj of fly (VB with will/MD as aux, so future). Luk is a conj of Alisa, which makes both of them subjects, and from → Rome depends on fly.
- **Sentence 2.** Here Alisa is the object (dobj) of visit, and the flight is from Athens.
- **Sentence 4.** The subject is the pronoun They and Alisa isn't in the sentence at all. The flight is from New York.

- **Sentence 5.** This was the one I was most unsure about, because it has both Alisa and flown from Rome. But the nsubj of flown is John, and Alisa is the nsubj of received in the subordinate clause (before Alisa received…). A plain keyword search would have accepted this sentence, while the dependency check rejects it.

9. Multilingual NLP: Swedish

Model: sv_core_news_sm. Text:

"Umeå universitet grundades 1965 och är i dag ett av Sveriges största lärosäten. Många studenter flyttar till Umeå för att studera språkteknologi, trots att vintrarna är långa och kalla."

("Umeå University was founded in 1965 and is today one of Sweden's largest institutions of higher education. Many students move to Umeå to study language technology, even though the winters are long and cold.")

Tokenization

`Umeå | universitet | grundades | 1965 | och | är | i | dag | ett | av | Sveriges | största | lärosäten | . | Många | studenter | flyttar | till | Umeå | för | att | studera | språkteknologi | , | trots | att | vintrarna | är | långa | och | kalla | .`

That gives 32 tokens, and I couldn't find any mistakes in the tokenization. å, ä and ö cause no problems. Swedish doesn't use apostrophe clitics, so the genitive Sveriges stays one token (compare Smith + 's in English) and the only thing split off is the punctuation.

POS tagging and dependency parsing (selected tokens)

Token	Lemma	UPOS	XPOS (SUC)	dep → head	Comment
universitet	universitet	NOUN	NN|NEU|SIN|IND|NOM	nsubj:pass → grundades	Correct
grundades	grundades	VERB	VB|PRT|SFO	ROOT	s-passive recognised: Voice=Pass, Tense=Past. Lemma should be grunda.
i dag	i / dag	ADP / NOUN	PP / NN	advmod → ett / fixed → i	Multiword expression correctly marked fixed
ett	en	DET	DT|NEU|SIN|IND	conj → grundades	Predicate "ett (av…)" with är as cop (UD style)
Sveriges	Sverige	PROPN	PM|GEN	nmod:poss → lärosäten	Correct genitive
största	stor	ADJ	JJ|SUV|…	amod → lärosäten	Superlative (Degree=Sup), lemma correct
lärosäten	lärosät	NOUN	NN|UTR|SIN|DEF|NOM	nmod → ett	Wrong: it is plural indefinite neuter, lemma lärosäte
flyttar	flytta	VERB	VB|PRS|AKT	ROOT	Present, active; subject studenter
för att studera		ADP PART VERB	PP IE VB|INF|AKT	mark, mark, advcl → flyttar	Purpose clause, correct
trots att		ADP SCONJ	PP SN	mark → långa / fixed	Concessive subordinator, correct (compare although in Task 6)
långa	lång	ADJ	JJ|POS|…|PLU	advcl → flyttar	Head of the concessive clause, är = cop
kalla	kalla	VERB	VB|INF|AKT	conj → långa	Wrong: the adjective "cold" is read as the verb kalla "to call"
Umeå (1st)	umeå	PROPN	PM|GEN	nmod:poss → universitet	Wrong: Umeå universitet is a name, not a genitive

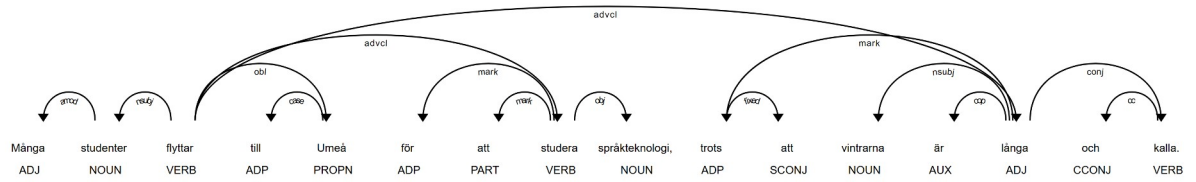


Figure 3. Dependency tree of the second Swedish sentence (sv_core_news_sm).

Analysis

Tagsets. UPOS is the same as for English (NOUN, VERB, ADJ …), which makes it easy to compare the two languages. The Swedish XPOS tags come from the SUC tagset, which is a lot richer than Penn Treebank. It encodes gender (UTR/NEU), number, definiteness, case, tense and voice, e.g. NN|UTR|PLU|DEF|NOM, since Swedish has more morphology to mark.

Syntax. The Swedish model uses Universal Dependencies labels (nsubj:pass, obl, case, cop, fixed), whereas the English model uses ClearNLP labels (nsubjpass, prep/pobj). The overall structure is right for both sentences. The roots are grundades and flyttar and the subjects universitet and studenter, the purpose clause för att studera and the concessive trots att … clause are both advcl, and the multiword expressions i dag and trots att are labelled fixed. The s-passive grundades gets Voice=Pass and an nsubj:pass, while English needs an extra word for the same thing (was founded, with auxpass).

Errors. The small model does make a fair number of mistakes. The most obvious one is kalla ("cold") tagged as a verb, because it's spelled the same as the infinitive kalla "to call", the same kind of problem as in Task 4. lärosäten is analysed as singular definite and gets the wrong lemma, and Umeå in the name Umeå universitet is tagged as a genitive. Some lemmas aren't reduced either (grundades, vintrarna). The Swedish model is trained on a lot less data than the English one, so I'm not that surprised. I would guess the larger sv_core_news_lg fixes some of these, but I haven't tried it.

Appendix: Complete program output

```
================================================================================
TASK 1: Tokenization with spaCy
================================================================================
Number of tokens: 46

['On', 'Jan.', '15', ',', '2024', ',', 'Dr.', 'Smith', "'s", 'research', 'on', 'AI', '-', 'generated', 'art', 'sparked', 'debates',
'on', "'", 'true', "'", 'creativity', '.', 'While', 'some', 'critics', 'call', 'it', 'mere', 'imitation', ',', 'others', 'see',
'it', 'as', 'a', '$', '10', 'million', 'revolution', 'in', 'human', '-', 'AI', 'collaboration', '!']

i    text         is_punct  like_num  is_currency  is_stop  whitespace_after
---  -----------  --------  --------  -----------  -------  ----------------
0    On           False     False     False        True     ' '
1    Jan.         False     False     False        False    ' '
2    15           False     True      False        False    ''
3    ,            True      False     False        False    ' '
4    2024         False     True      False        False    ''
5    ,            True      False     False        False    ' '
6    Dr.          False     False     False        False    ' '
7    Smith        False     False     False        False    ''
8    's           False     False     False        True     ' '
9    research     False     False     False        False    ' '
10   on           False     False     False        True     ' '
11   AI           False     False     False        False    ''
12   -            True      False     False        False    ''
13   generated    False     False     False        False    ' '
14   art          False     False     False        False    ' '
15   sparked      False     False     False        False    ' '
16   debates      False     False     False        False    ' '
17   on           False     False     False        True     ' '
18   '            True      False     False        False    ''
19   true         False     False     False        False    ''
20   '            True      False     False        False    ' '
```

```
21   creativity    False     False     False        False    ''
22   .             True      False     False        False    ' '
23   While         False     False     False        True     ' '
24   some          False     False     False        True     ' '
25   critics       False     False     False        False    ' '
26   call          False     False     False        True     ' '
27   it            False     False     False        True     ' '
28   mere          False     False     False        False    ' '
29   imitation     False     False     False        False    ''
30   ,             True      False     False        False    ' '
31   others        False     False     False        True     ' '
32   see           False     False     False        True     ' '
33   it            False     False     False        True     ' '
34   as            False     False     False        True     ' '
35   a             False     False     False        True     ' '
36   $             False     False     True         False    ''
37   10            False     True      False        False    ' '
38   million       False     True      False        False    ' '
39   revolution    False     False     False        False    ' '
40   in            False     False     False        True     ' '
41   human         False     False     False        False    ''
42   –             True      False     False        False    ''
43   AI            False     False     False        False    ' '
44   collaboration  False     False     False        False    ''
45   !             True      False     False        False    ''

============================================================================
TASK 2: Tokens that are not numbers, number words or stop words
============================================================================
Kept (28 tokens):
['Jan.', ',', ',', 'Dr.', 'Smith', 'research', 'AI', '-', 'generated', 'art', 'sparked', 'debates', "'", 'true', "'", 'creativity',
'.', 'critics', 'mere', 'imitation', ',', '$', 'revolution', 'human', '–', 'AI', 'collaboration', '!']

Removed (18 tokens):
[('On', 'is_stop'), ('15', 'like_num'), ('2024', 'like_num'), ("'s", 'is_stop'), ('on', 'is_stop'), ('on', 'is_stop'), ('While',
'is_stop'), ('some', 'is_stop'), ('call', 'is_stop'), ('it', 'is_stop'), ('others', 'is_stop'), ('see', 'is_stop'), ('it',
'is_stop'), ('as', 'is_stop'), ('a', 'is_stop'), ('10', 'like_num'), ('million', 'like_num'), ('in', 'is_stop')]

Variant that also removes punctuation/currency (18 tokens):
['Jan.', 'Dr.', 'Smith', 'research', 'AI', 'generated', 'art', 'sparked', 'debates', 'true', 'creativity', 'critics', 'mere',
'imitation', 'revolution', 'human', 'AI', 'collaboration']

like_num check on number words: [('one', True), ('two', True), ('first', True), ('third', True), ('ten', True), ('hundred', True),
('million', True), ('2024', True), ('15', True), ('3.5', True)]

============================================================================
TASK 3: spaCy tokenizer vs. Python's split()
============================================================================
Text: Hello, world! Isn't this state-of-the-art? I'm sure it is. Don't you think so, Dr. Brown? We'll see: prices rose 3.5% in the
U.S.

split() -> 23 tokens:
['Hello,', 'world!', "Isn't", 'this', 'state-of-the-art?', "I'm", 'sure', 'it', 'is.', "Don't", 'you', 'think', 'so,', 'Dr.',
'Brown?', "We'll", 'see:', 'prices', 'rose', '3.5%', 'in', 'the', 'U.S.']

spaCy   -> 41 tokens:
['Hello', ',', 'world', '!', 'Is', "n't", 'this', 'state', '-', 'of', '-', 'the', '-', 'art', '?', 'I', "'m", 'sure', 'it', 'is',
'.', 'Do', "n't", 'you', 'think', 'so', ',', 'Dr.', 'Brown', '?', 'We', "'ll", 'see', ':', 'prices', 'rose', '3.5', '%', 'in',
'the', 'U.S.']

How each approach handles specific phenomena:
  world!             split(): ['world!']             spaCy: ['world', '!']
  so,                split(): ['so,']                spaCy: ['so', ',']
  Isn't              split(): ["Isn't"]              spaCy: ['Is', "n't"]
  I'm                split(): ["I'm"]                spaCy: ['I', "'m"]
  don't              split(): ["don't"]              spaCy: ['do', "n't"]
  state-of-the-art   split(): ['state-of-the-art']   spaCy: ['state', '-', 'of', '-', 'the', '-', 'art']
  AI-generated       split(): ['AI-generated']       spaCy: ['AI', '-', 'generated']
  Dr.                split(): ['Dr.']                spaCy: ['Dr.']
  U.S.               split(): ['U.S.']               spaCy: ['U.S.']
  3.5%               split(): ['3.5%']               spaCy: ['3.5', '%']
  $10                split(): ['$10']                spaCy: ['$', '10']

Tags with spaCy tokens:
[('I', 'PRP'), ("'m", 'VBP'), ('sure', 'JJ'), ('it', 'PRP'), ('is', 'VBZ'), ('.', '.'), ('Do', 'VBP'), ("n't", 'RB'), ('you',
'PRP'), ('think', 'VB'), ('so', 'RB'), (',', ','), ('Dr.', 'NNP'), ('Brown', 'NNP'), ('?', '.')]
Tags with split() tokens:
[("I'm", 'PRP'), ('sure', 'JJ'), ('it', 'PRP'), ('is.', 'UH'), ("Don't", 'RB'), ('you', 'PRP'), ('think', 'VBP'), ('so,', 'NN'),
('Dr.', 'NNP'), ('Brown?', 'NN')]

============================================================================
TASK 4: UPOS (token.pos_) vs. XPOS (token.tag_)
============================================================================
```

Sentence 1: The quick brown fox jumps over the lazy dog, but why does it always pick on that poor dog?

text	UPOS	XPOS	XPOS meaning	dep	head
The	DET	DT	determiner	det	fox
quick	ADJ	JJ	adjective (English), other noun-modifi	amod	fox
brown	ADJ	JJ	adjective (English), other noun-modifi	amod	fox
fox	NOUN	NN	noun, singular or mass	nsubj	jumps
jumps	VERB	VBZ	verb, 3rd person singular present	ROOT	jumps
over	ADP	IN	conjunction, subordinating or preposit	prep	jumps
the	DET	DT	determiner	det	dog
lazy	ADJ	JJ	adjective (English), other noun-modifi	amod	dog
dog	NOUN	NN	noun, singular or mass	pobj	over
,	PUNCT	,	punctuation mark, comma	punct	jumps
but	CCONJ	CC	conjunction, coordinating	cc	jumps
why	SCONJ	WRB	wh-adverb	advmod	pick
does	AUX	VBZ	verb, 3rd person singular present	aux	pick
it	PRON	PRP	pronoun, personal	nsubj	pick
always	ADV	RB	adverb	advmod	pick
pick	VERB	VB	verb, base form	conj	jumps
on	ADP	IN	conjunction, subordinating or preposit	prep	pick
that	DET	DT	determiner	det	dog
poor	ADJ	JJ	adjective (English), other noun-modifi	amod	dog
dog	NOUN	NN	noun, singular or mass	pobj	on
?	PUNCT	.	punctuation mark, sentence closer	punct	pick

Sentence 2: She saw the man with binoculars.

text	UPOS	XPOS	XPOS meaning	dep	head
She	PRON	PRP	pronoun, personal	nsubj	saw
saw	VERB	VBD	verb, past tense	ROOT	saw
the	DET	DT	determiner	det	man
man	NOUN	NN	noun, singular or mass	dobj	saw
with	ADP	IN	conjunction, subordinating or preposit	prep	man
binoculars	NOUN	NNS	noun, plural	pobj	with
.	PUNCT	.	punctuation mark, sentence closer	punct	saw

Sentence 3: Time flies like an arrow; fruit flies like a banana.

text	UPOS	XPOS	XPOS meaning	dep	head
Time	NOUN	NN	noun, singular or mass	nsubj	flies
flies	VERB	VBZ	verb, 3rd person singular present	ROOT	flies
like	ADP	IN	conjunction, subordinating or preposit	prep	flies
an	DET	DT	determiner	det	arrow
arrow	NOUN	NN	noun, singular or mass	pobj	like
;	PUNCT	:	punctuation mark, colon or ellipsis	punct	flies
fruit	NOUN	NN	noun, singular or mass	compound	flies
flies	NOUN	NNS	noun, plural	conj	flies
like	ADP	IN	conjunction, subordinating or preposit	prep	flies
a	DET	DT	determiner	det	banana
banana	NOUN	NN	noun, singular or mass	pobj	like
.	PUNCT	.	punctuation mark, sentence closer	punct	flies

Same word form, different context:
  flies [('flies', 'VERB', 'VBZ', 'ROOT'), ('flies', 'NOUN', 'NNS', 'conj')]
  like   [('like', 'ADP', 'IN', 'prep'), ('like', 'ADP', 'IN', 'prep')]
  saw   [('saw', 'VERB', 'VBD', 'ROOT')]
  that   [('that', 'DET', 'DT', 'det')]
  dog   [('dog', 'NOUN', 'NN', 'pobj'), ('dog', 'NOUN', 'NN', 'pobj')]

PP attachment: 'with' is attached to 'man' (dep=prep)

Saved dependency visualisation to C:\Users\vavyl\Desktop\New folder (2)\output\task4_time_flies.svg

Comparison en_core_web_sm vs. en_core_web_md on the ambiguous sentences:
  sm: She/PRON/PRP->saw saw/VERB/VBD->saw the/DET/DT->man man/NOUN/NN->saw with/ADP/IN->man binoculars/NOUN/NNS->with
  md: She/PRON/PRP->saw saw/VERB/VBD->saw the/DET/DT->man man/NOUN/NN->saw with/ADP/IN->saw binoculars/NOUN/NNS->with
  sm: Time/NOUN/NN->flies flies/VERB/VBZ->flies like/ADP/IN->flies an/DET/DT->arrow arrow/NOUN/NN->like fruit/NOUN/NN->flies flies/NOUN/NNS->flies like/ADP/IN->flies a/DET/DT->banana banana/NOUN/NN->like
  md: Time/NOUN/NN->flies flies/VERB/VBZ->flies like/ADP/IN->flies an/DET/DT->arrow arrow/NOUN/NN->like fruit/NOUN/NN->flies flies/VERB/VBZ->flies like/ADP/IN->flies a/DET/DT->banana banana/NOUN/NN->like

==
TASK 5: Travel queries - which ones are about a planned future flight?
==

Query 1: We flew to Rome 3 days ago. I still haven't received the bill, please send it ASAP.

text	POS	TAG	dep	head	morph
We	PRON	PRP	nsubj	flew	Case=Nom|Number=Plur|Person=1|PronType=Prs
flew	VERB	VBD	ROOT	flew	Tense=Past|VerbForm=Fin
to	ADP	IN	prep	flew	
Rome	PROPN	NNP	pobj	to	Number=Sing

```
3             NUM    CD    nummod    days        NumType=Card
days          NOUN   NNS   npadvmod  ago         Number=Plur
ago           ADV    RB    advmod    flew
.             PUNCT  .     punct     flew        PunctType=Peri
I             PRON   PRP   nsubj     received    Case=Nom|Number=Sing|Person=1|PronType=Prs
still         ADV    RB    advmod    received
have          AUX    VBP   aux       received    Mood=Ind|Tense=Pres|VerbForm=Fin
n't           PART   RB    neg       received    Polarity=Neg
received      VERB   VBN   ccomp     send        Aspect=Perf|Tense=Past|VerbForm=Part
the           DET    DT    det       bill        Definite=Def|PronType=Art
bill          NOUN   NN    dobj      received    Number=Sing
,             PUNCT  ,     punct     send        PunctType=Comm
please        INTJ   UH    intj      send
send          VERB   VB    ROOT      send        VerbForm=Inf
it            PRON   PRP   dobj      send        Case=Acc|Gender=Neut|Number=Sing|Person=3|PronType=Prs
ASAP          PROPN  NNP   advmod    send        Number=Sing
.             PUNCT  .     punct     send        PunctType=Peri
  -> 'flew': PAST  [simple past (VBD), Tense=Past]

Query 2: I have flown to Rome this morning and forgot my laptop on the plane. Can you please connect me to lost and found?
text          POS    TAG   dep       head        morph
------------  -----  ----  --------  ----------  ---------------------------------------
I             PRON   PRP   nsubj     flown       Case=Nom|Number=Sing|Person=1|PronType=Prs
have          AUX    VBP   aux       flown       Mood=Ind|Tense=Pres|VerbForm=Fin
flown         VERB   VBN   ROOT      flown       Aspect=Perf|Tense=Past|VerbForm=Part
to            ADP    IN    prep      flown
Rome          PROPN  NNP   pobj      to          Number=Sing
this          DET    DT    det       morning     Number=Sing|PronType=Dem
morning       NOUN   NN    npadvmod  flown       Number=Sing
and           CCONJ  CC    cc        flown       ConjType=Cmp
forgot        VERB   VBD   conj      flown       Tense=Past|VerbForm=Fin
my            PRON   PRP$  poss      laptop      Number=Sing|Person=1|Poss=Yes|PronType=Prs
laptop        NOUN   NN    dobj      forgot      Number=Sing
on            ADP    IN    prep      forgot
the           DET    DT    det       plane       Definite=Def|PronType=Art
plane         NOUN   NN    pobj      on          Number=Sing
.             PUNCT  .     punct     flown       PunctType=Peri
Can           AUX    MD    aux       connect     VerbForm=Fin
you           PRON   PRP   nsubj     connect     Case=Nom|Person=2|PronType=Prs
please        INTJ   UH    intj      connect
connect       VERB   VB    ROOT      connect     VerbForm=Inf
me            PRON   PRP   dobj      connect     Case=Acc|Number=Sing|Person=1|PronType=Prs
to            PART   TO    aux       lost
lost          VERB   VB    xcomp     connect     VerbForm=Inf
and           CCONJ  CC    cc        lost        ConjType=Cmp
found         VERB   VBN   conj      lost        Aspect=Perf|Tense=Past|VerbForm=Part
?             PUNCT  .     punct     connect     PunctType=Peri
  -> 'have flown': PAST (present perfect)  [aux 'have' + past participle (VBN), Tense=Past]

Query 3: I'm flying to Rome next week. Can you check flight availability?
text          POS    TAG   dep       head        morph
------------  -----  ----  --------  ----------  ---------------------------------------
I             PRON   PRP   nsubj     flying      Case=Nom|Number=Sing|Person=1|PronType=Prs
'm            AUX    VBP   aux       flying      Mood=Ind|Number=Sing|Person=1|Tense=Pres|VerbForm=Fin
flying        VERB   VBG   ROOT      flying      Aspect=Prog|Tense=Pres|VerbForm=Part
to            ADP    IN    prep      flying
Rome          PROPN  NNP   pobj      to          Number=Sing
next          ADJ    JJ    amod      week        Degree=Pos
week          NOUN   NN    npadvmod  flying      Number=Sing
.             PUNCT  .     punct     flying      PunctType=Peri
Can           AUX    MD    aux       check       VerbForm=Fin
you           PRON   PRP   nsubj     check       Case=Nom|Person=2|PronType=Prs
check         VERB   VB    ROOT      check       VerbForm=Inf
flight        NOUN   NN    compound  availability  Number=Sing
availability  NOUN   NN    dobj      check       Number=Sing
?             PUNCT  .     punct     check       PunctType=Peri
  -> ''m flying': FUTURE (planned)  [aux 'be' (VBP) + VBG, Aspect=Prog, with future time expression ('next ...')]

Query 4: We need to fly to Rome. Can you check flights for next Tuesday?
text          POS    TAG   dep       head        morph
------------  -----  ----  --------  ----------  ---------------------------------------
We            PRON   PRP   nsubj     need        Case=Nom|Number=Plur|Person=1|PronType=Prs
need          VERB   VBP   ROOT      need        Tense=Pres|VerbForm=Fin
to            PART   TO    aux       fly
fly           VERB   VB    xcomp     need        VerbForm=Inf
to            ADP    IN    prep      fly
Rome          PROPN  NNP   pobj      to          Number=Sing
.             PUNCT  .     punct     need        PunctType=Peri
Can           AUX    MD    aux       check       VerbForm=Fin
you           PRON   PRP   nsubj     check       Case=Nom|Person=2|PronType=Prs
check         VERB   VB    ROOT      check       VerbForm=Inf
flights       NOUN   NNS   dobj      check       Number=Plur
for           ADP    IN    prep      flights
```

```
next          ADJ    JJ    amod      Tuesday     Degree=Pos
Tuesday       PROPN  NNP   pobj      for         Number=Sing
?             PUNCT  .     punct     check       PunctType=Peri
  -> 'to fly': FUTURE (intent)  [xcomp of 'need' (VBP), 'to' (TO/aux) + base form (VB)]

Query 5: I will fly to Rome next week. Can you check the flights?
text          POS    TAG   dep       head        morph
------------  -----  ----  --------  ----------  ---------------------------------------
I             PRON   PRP   nsubj     fly         Case=Nom|Number=Sing|Person=1|PronType=Prs
will          AUX    MD    aux       fly         VerbForm=Fin
fly           VERB   VB    ROOT      fly         VerbForm=Inf
to            ADP    IN    prep      fly
Rome          PROPN  NNP   pobj      to          Number=Sing
next          ADJ    JJ    amod      week        Degree=Pos
week          NOUN   NN    npadvmod  fly         Number=Sing
.             PUNCT  .     punct     fly         PunctType=Peri
Can           AUX    MD    aux       check       VerbForm=Fin
you           PRON   PRP   nsubj     check       Case=Nom|Person=2|PronType=Prs
check         VERB   VB    ROOT      check       VerbForm=Inf
the           DET    DT    det       flights     Definite=Def|PronType=Art
flights       NOUN   NNS   dobj      check       Number=Plur
?             PUNCT  .     punct     check       PunctType=Peri
  -> 'will fly': FUTURE  [modal aux 'will' (MD) + base form (VB)]

Query 6: They will fly to Vienna next Monday. Can you check the flights?
text          POS    TAG   dep       head        morph
------------  -----  ----  --------  ----------  ---------------------------------------
They          PRON   PRP   nsubj     fly         Case=Nom|Number=Plur|Person=3|PronType=Prs
will          AUX    MD    aux       fly         VerbForm=Fin
fly           VERB   VB    ROOT      fly         VerbForm=Inf
to            ADP    IN    prep      fly
Vienna        PROPN  NNP   pobj      to          Number=Sing
next          ADJ    JJ    amod      Monday      Degree=Pos
Monday        PROPN  NNP   npadvmod  fly         Number=Sing
.             PUNCT  .     punct     fly         PunctType=Peri
Can           AUX    MD    aux       check       VerbForm=Fin
you           PRON   PRP   nsubj     check       Case=Nom|Person=2|PronType=Prs
check         VERB   VB    ROOT      check       VerbForm=Inf
the           DET    DT    det       flights     Definite=Def|PronType=Art
flights       NOUN   NNS   dobj      check       Number=Plur
?             PUNCT  .     punct     check       PunctType=Peri
  -> 'will fly': FUTURE  [modal aux 'will' (MD) + base form (VB)]

Queries about a planned future flight (booking intent): [3, 4, 5, 6]

================================================================================
TASK 6: Dependency tree
================================================================================
i   text       POS    dep     explanation                head       children
--  ---------  -----  ------  -------------------------  ---------  ------------------------------
0   Although   SCONJ  mark    marker                     was        []
1   she        PRON   nsubj   nominal subject            was        []
2   was        AUX    advcl   adverbial clause modifier  finished   ['Although', 'she', 'tired']
3   tired      ADJ    acomp   adjectival complement      was        []
4   ,          PUNCT  punct   punctuation                finished   []
5   she        PRON   nsubj   nominal subject            finished   []
6   finished   VERB   ROOT    root                       finished   ['was', ',', 'she', 'work', 'before', '.']
7   her        PRON   poss    possession modifier        work       []
8   work       NOUN   dobj    direct object              finished   ['her']
9   before     ADP    prep    prepositional modifier     finished   ['midnight']
10  midnight   NOUN   pobj    object of preposition      before     []
11  .          PUNCT  punct   punctuation                finished   []

Main clause verb (ROOT): finished
Subject of main clause:  ['she']
Object of main clause:   ['her work']

'Although': dep=mark (marker), head='was'
'was': dep=advcl (adverbial clause modifier), head='finished'
Subordinate clause: Although she was tired

Tree (indented):
finished [ROOT, VERB]
    was [advcl, AUX]
        Although [mark, SCONJ]
        she [nsubj, PRON]
        tired [acomp, ADJ]
    , [punct, PUNCT]
    she [nsubj, PRON]
    work [dobj, NOUN]
        her [poss, PRON]
    before [prep, ADP]
        midnight [pobj, NOUN]
```

```
    . [punct, PUNCT]

Saved dependency visualisation to C:\Users\vavyl\Desktop\New folder (2)\output\task6_dependency_tree.svg

================================================================================
TASK 7: Passive voice
================================================================================

John bought a ticket for the concert yesterday.
  deps: [('John', 'nsubj', 'NNP'), ('bought', 'ROOT', 'VBD'), ('a', 'det', 'DT'), ('ticket', 'dobj', 'NN'), ('for', 'prep', 'IN'),
('the', 'det', 'DT'), ('concert', 'pobj', 'NN'), ('yesterday', 'npadvmod', 'NN')]
  -> ACTIVE voice (no nsubjpass/auxpass)

Alice helped her friend with the homework.
  deps: [('Alice', 'nsubj', 'NNP'), ('helped', 'ROOT', 'VBD'), ('her', 'poss', 'PRP$'), ('friend', 'dobj', 'NN'), ('with', 'prep',
'IN'), ('the', 'det', 'DT'), ('homework', 'pobj', 'NN')]
  -> ACTIVE voice (no nsubjpass/auxpass)

The team completed the project on time.
  deps: [('The', 'det', 'DT'), ('team', 'nsubj', 'NN'), ('completed', 'ROOT', 'VBD'), ('the', 'det', 'DT'), ('project', 'dobj',
'NN'), ('on', 'prep', 'IN'), ('time', 'pobj', 'NN')]
  -> ACTIVE voice (no nsubjpass/auxpass)

The problem was solved quickly by Mark.
  deps: [('The', 'det', 'DT'), ('problem', 'nsubjpass', 'NN'), ('was', 'auxpass', 'VBD'), ('solved', 'ROOT', 'VBN'), ('quickly',
'advmod', 'RB'), ('by', 'agent', 'IN'), ('Mark', 'pobj', 'NNP')]
  -> PASSIVE voice: auxpass='was' (VBD) + 'solved' (VBN)
     patient (nsubjpass): The problem
     agent   (agent->pobj): Mark
     tense: past  (aux morph: Mood=Ind|Number=Sing|Person=3|Tense=Past|VerbForm=Fin)

A new product was launched by the company last week.
  deps: [('A', 'det', 'DT'), ('new', 'amod', 'JJ'), ('product', 'nsubjpass', 'NN'), ('was', 'auxpass', 'VBD'), ('launched', 'ROOT',
'VBN'), ('by', 'agent', 'IN'), ('the', 'det', 'DT'), ('company', 'pobj', 'NN'), ('last', 'amod', 'JJ'), ('week', 'npadvmod', 'NN')]
  -> PASSIVE voice: auxpass='was' (VBD) + 'launched' (VBN)
     patient (nsubjpass): A new product
     agent   (agent->pobj): the company
     tense: past  (aux morph: Mood=Ind|Number=Sing|Person=3|Tense=Past|VerbForm=Fin)

Flowers were planted in the garden by Sarah.
  deps: [('Flowers', 'nsubjpass', 'NNS'), ('were', 'auxpass', 'VBD'), ('planted', 'ROOT', 'VBN'), ('in', 'prep', 'IN'), ('the',
'det', 'DT'), ('garden', 'pobj', 'NN'), ('by', 'agent', 'IN'), ('Sarah', 'pobj', 'NNP')]
  -> PASSIVE voice: auxpass='were' (VBD) + 'planted' (VBN)
     patient (nsubjpass): Flowers
     agent   (agent->pobj): Sarah
     tense: past  (aux morph: Mood=Ind|Tense=Past|VerbForm=Fin)

The decision was made yesterday.
  deps: [('The', 'det', 'DT'), ('decision', 'nsubjpass', 'NN'), ('was', 'auxpass', 'VBD'), ('made', 'ROOT', 'VBN'), ('yesterday',
'npadvmod', 'NN')]
  -> PASSIVE voice: auxpass='was' (VBD) + 'made' (VBN)
     patient (nsubjpass): The decision
     agent   (agent->pobj): (not expressed - implicit agent)
     tense: past  (aux morph: Mood=Ind|Number=Sing|Person=3|Tense=Past|VerbForm=Fin)

================================================================================
TASK 8: Sentences where Alisa is the subject of flying from Rome
================================================================================

1. Alisa flew from Rome to Vienna three days ago.
  verb: flew (POS=VERB, TAG=VBD, morph=Tense=Past|VerbForm=Fin)
  subjects: [('Alisa', 'nsubj', 'PROPN')]
  from:     Rome
  -> MATCH
  'Alisa': POS=PROPN, TAG=NNP, dep=nsubj, head='flew'

2. My parents have flown from Athens to Rome this morning to visit Alisa.
  verb: have flown (POS=VERB, TAG=VBN, morph=Aspect=Perf|Tense=Past|VerbForm=Part)
  subjects: [('parents', 'nsubj', 'NOUN')]
  from:     Athens
  -> no match
  'Alisa': POS=PROPN, TAG=NNP, dep=dobj, head='visit'

3. Alisa and Luk will fly from Rome to Paris next week for a conference.
  verb: will fly (POS=VERB, TAG=VB, morph=VerbForm=Inf)
  subjects: [('Alisa', 'nsubj', 'PROPN'), ('Luk', 'conj', 'PROPN')]
  from:     Rome
  -> MATCH
  'Alisa': POS=PROPN, TAG=NNP, dep=nsubj, head='fly'

4. They are flying from New York to Rome tomorrow.
  verb: are flying (POS=VERB, TAG=VBG, morph=Aspect=Prog|Tense=Pres|VerbForm=Part)
  subjects: [('They', 'nsubj', 'PRON')]
```

```
  from:     New York
  -> no match

5. John had flown from Rome to London before Alisa received the urgent call.
  verb: had flown (POS=VERB, TAG=VBN, morph=Aspect=Perf|Tense=Past|VerbForm=Part)
  subjects: [('John', 'nsubj', 'PROPN')]
  from:     Rome
  -> no match
  'Alisa': POS=PROPN, TAG=NNP, dep=nsubj, head='received'

Matching sentences: [1, 3]

================================================================================
TASK 9: Multilingual NLP - Swedish (sv_core_news_sm)
================================================================================
Text: Umeå universitet grundades 1965 och är i dag ett av Sveriges största lärosäten. Många studenter flyttar till Umeå för att
studera språkteknologi, trots att vintrarna är långa och kalla.

1) Tokens:
['Umeå', 'universitet', 'grundades', '1965', 'och', 'är', 'i', 'dag', 'ett', 'av', 'Sveriges', 'största', 'lärosäten', '.',
'Många', 'studenter', 'flyttar', 'till', 'Umeå', 'för', 'att', 'studera', 'språkteknologi', ',', 'trots', 'att', 'vintrarna', 'är',
'långa', 'och', 'kalla', '.']

2) POS tagging and 3) dependency parsing:
text           lemma          UPOS    XPOS (SUC)              dep          head          morph
-------------  -------------  ------  ----------------------  -----------  ------------
-----------------------------------------------
Umeå           umeå           PROPN   PM|GEN                  nmod:poss    universitet   Case=Gen
universitet    universitet    NOUN    NN|NEU|SIN|IND|NOM      nsubj:pass   grundades     Case=Nom|Definite=Ind|Gender=Neut|
Number=Sing
grundades      grundades      VERB    VB|PRT|SFO              ROOT         grundades     Mood=Ind|Tense=Past|VerbForm=Fin|Voice=Pass
1965           1965           NUM     RG|NOM                  nummod       grundades     Case=Nom|NumType=Card
och            och            CCONJ   KN                      cc           ett
är             vara           AUX     VB|PRS|AKT              cop          ett           Mood=Ind|Tense=Pres|VerbForm=Fin|Voice=Act
i              i              ADP     PP                      advmod       ett
dag            dag            NOUN    NN|UTR|SIN|IND|NOM      fixed        i             Case=Nom|Definite=Ind|Gender=Com|
Number=Sing
ett            en             DET     DT|NEU|SIN|IND          conj         grundades     Definite=Ind|Gender=Neut|Number=Sing|
PronType=Art
av             av             ADP     PP                      case         lärosäten
Sveriges       Sverige        PROPN   PM|GEN                  nmod:poss    lärosäten     Case=Gen
största        stor           ADJ     JJ|SUV|UTR/NEU|SIN/PLU|DEF|NOM  amod         lärosäten     Case=Nom|Definite=Def|Degree=Sup
lärosäten      lärosät        NOUN    NN|UTR|SIN|DEF|NOM      nmod         ett           Case=Nom|Definite=Def|Gender=Com|
Number=Sing
.              .              PUNCT   MAD                     punct        grundades
Många          många          ADJ     JJ|POS|UTR/NEU|PLU|IND/DEF|NOM  amod         studenter     Case=Nom|Degree=Pos|Number=Plur
studenter      student        NOUN    NN|UTR|PLU|IND|NOM      nsubj        flyttar       Case=Nom|Definite=Ind|Gender=Com|
Number=Plur
flyttar        flytta         VERB    VB|PRS|AKT              ROOT         flyttar       Mood=Ind|Tense=Pres|VerbForm=Fin|Voice=Act
till           till           ADP     PP                      case         Umeå
Umeå           Umeå           PROPN   PM|NOM                  obl          flyttar       Case=Nom
för            för            ADP     PP                      mark         studera
att            att            PART    IE                      mark         studera
studera        studera        VERB    VB|INF|AKT              advcl        flyttar       VerbForm=Inf|Voice=Act
språkteknologi  språkteknologi  NOUN    NN|UTR|SIN|IND|NOM      obj          studera       Case=Nom|Definite=Ind|Gender=Com|
Number=Sing
,              ,              PUNCT   MID                     punct        flyttar
trots          trots          ADP     PP                      mark         långa
att            att            SCONJ   SN                      fixed        trots
vintrarna      vintrarna      NOUN    NN|UTR|PLU|DEF|NOM      nsubj        långa         Case=Nom|Definite=Def|Gender=Com|
Number=Plur
är             vara           AUX     VB|PRS|AKT              cop          långa         Mood=Ind|Tense=Pres|VerbForm=Fin|Voice=Act
långa          lång           ADJ     JJ|POS|UTR/NEU|PLU|IND/DEF|NOM  advcl        flyttar       Case=Nom|Degree=Pos|Number=Plur
och            och            CCONJ   KN                      cc           kalla
kalla          kalla          VERB    VB|INF|AKT              conj         långa         VerbForm=Inf|Voice=Act
.              .              PUNCT   MAD                     punct        flyttar

Sentence 1: ROOT='grundades' (VERB), subject=['universitet']

Sentence 2: ROOT='flyttar' (VERB), subject=['studenter']

Named entities: [('Umeå universitet', 'ORG'), ('Sveriges', 'LOC'), ('Umeå', 'LOC')]

Saved dependency visualisation to C:\Users\vavyl\Desktop\New folder (2)\output\task9_swedish_sentence1.svg

Saved dependency visualisation to C:\Users\vavyl\Desktop\New folder (2)\output\task9_swedish_sentence2.svg
```